

RECURSION, DICTIONARIES

(download slides and .py files and follow along!)

6.0001 LECTURE 6

QUIZ PREP

- a paper and an online component
- open book/notes
- not open Internet, not open computer
- start printing out whatever you may want to bring

LAST TIME

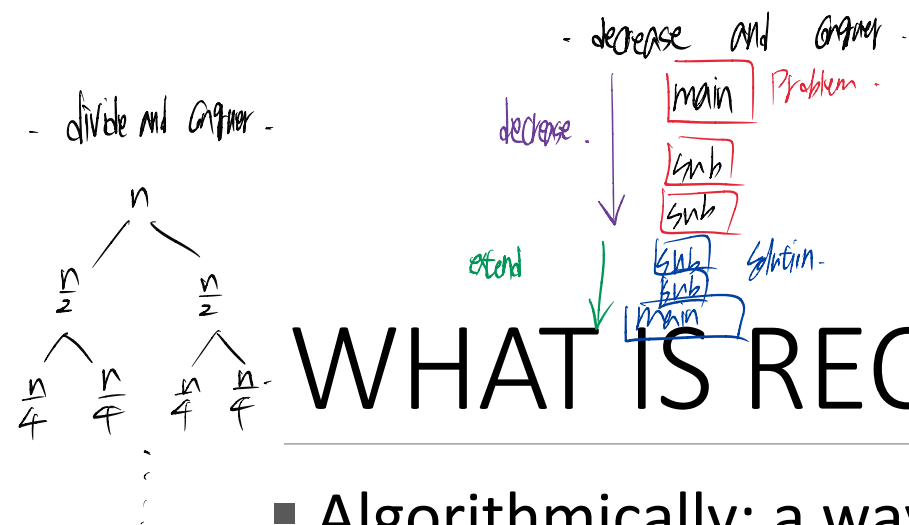
- tuples - immutable
- lists - mutable
- aliasing, cloning
- mutability side effects

TODAY

- recursion – divide/decrease and conquer
- dictionaries – another mutable object type

RECURSION

Recursion is the process of repeating items in a self-similar way.



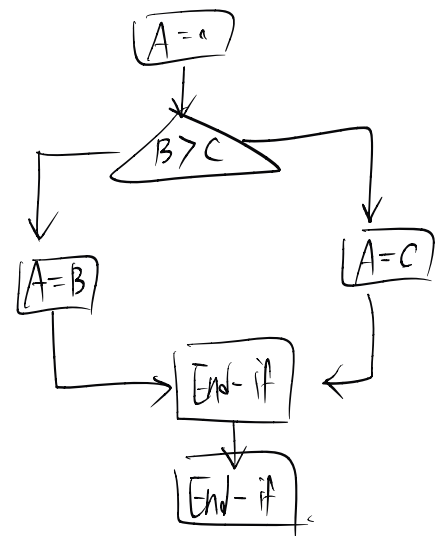
WHAT IS RECURSION?

- Algorithmically: a way to design solutions to problems by **divide-and-conquer** or **decrease-and-conquer**
 - reduce a problem to simpler versions of the same problem
- Semantically: a programming technique where a **function calls itself**
 - in programming, goal is to NOT have infinite recursion
 - must have **1 or more base cases** that are easy to solve
 - must solve the same problem on **some other input** with the goal of simplifying the larger problem input

ITERATIVE ALGORITHMS SO FAR

- looping constructs (`while` and `for` loops) lead to **iterative** algorithms
- can capture computation in a set of **state variables** that update on each iteration through loop

iterative algorithm



MULTIPLICATION – ITERATIVE SOLUTION

- “multiply $a * b$ ” is equivalent to “add a to itself b times”

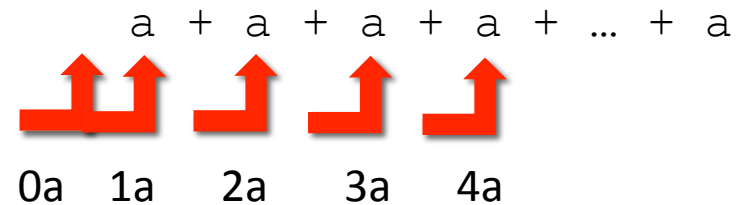
- capture **state** by

- an **iteration** number (i) starts at b

$i \leftarrow i - 1$ and stop when 0

- a current **value of computation** (result)

$\text{result} \leftarrow \text{result} + a$



```
def mult_iter(a, b):
```

```
    result = 0
```

```
    while b > 0:
```

```
        result += a
```

```
        b -= 1
```

```
    return result
```

iteration
current value of computation,
a running sum
current value of iteration variable

MULTIPLICATION – RECURSIVE SOLUTION

■ recursive step

- think how to reduce problem to a **simpler/smaller version** of same problem

$$\begin{aligned} a * b &= \underbrace{a + a + a + a + \dots + a}_{b \text{ times}} \\ &= a + \underbrace{a + a + a + \dots + a}_{b-1 \text{ times}} \\ &= a + \boxed{a * (b-1)} \end{aligned}$$

recursive reduction

■ base case

- keep reducing problem until reach a simple case that can be **solved directly**
- when $b = 1$, $a * b = a$

```
def mult(a, b):
```

```
    if b == 1:  
        return a
```

```
    else:  
        return a + mult(a, b-1)
```

$\Leftrightarrow a * (b-1)$

FACTORIAL

$$\begin{aligned} \text{Ex) } 5! &= 5 \cdot (5-1) \cdot (5-2) \cdot (5-3) \cdot (5-4) = 120 \\ &= 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 \\ &= 120 \end{aligned}$$

$$n! = n \cdot (n-1) \cdot (n-2) \cdot (n-3) \cdot \dots \cdot 1$$

- for what n do we know the factorial?

$n = 1 \quad \rightarrow \quad \text{if } n == 1:$
 $\quad \quad \quad \text{return } 1$ *base case*

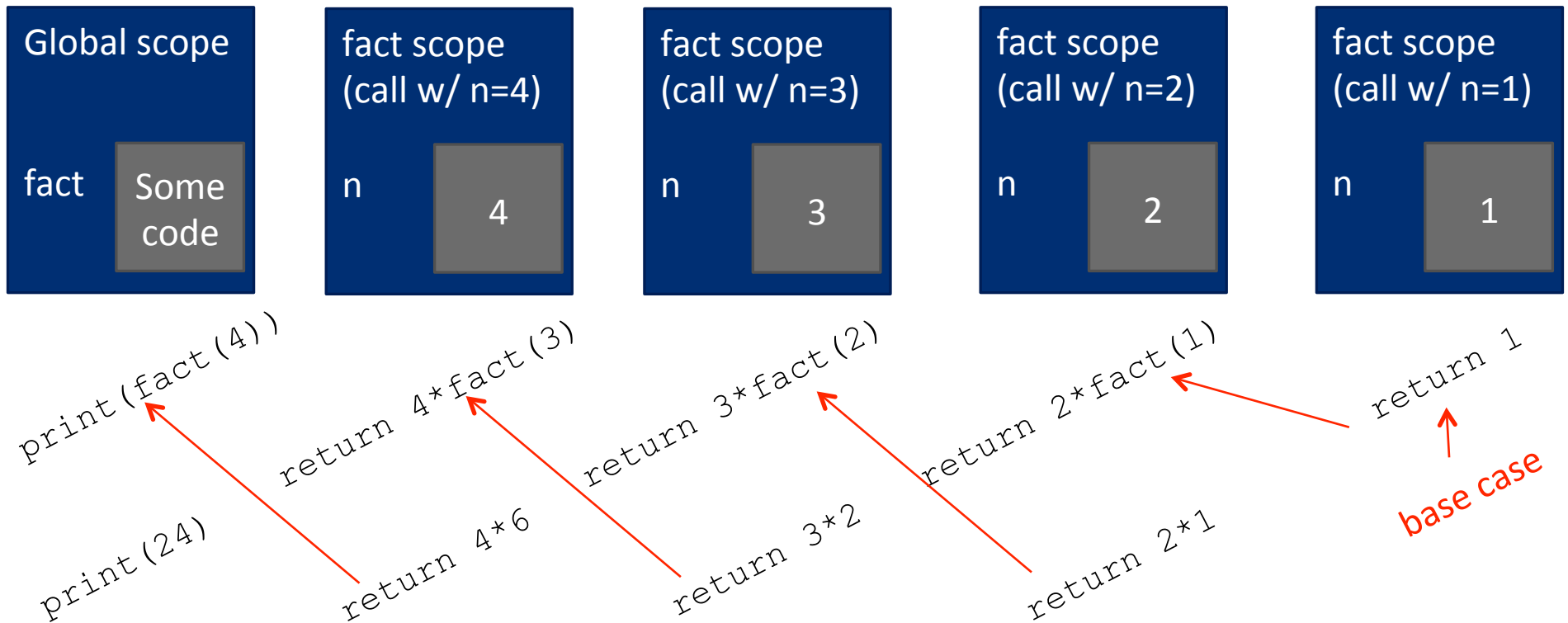
- how to reduce problem? Rewrite in terms of something simpler to reach base case

$n \cdot (n-1)! \quad \rightarrow \quad \text{else:}$
 $\quad \quad \quad \text{return } n \cdot \text{factorial}(n-1)$

recursive step

RECURSIVE FUNCTION SCOPE EXAMPLE

```
def fact(n):  
    if n == 1:  
        return 1  
    else:  
        return n*fact(n-1)  
  
print(fact(4))
```



SOME OBSERVATIONS

- each recursive call to a function creates its **own scope/environment**
- **bindings of variables** in a scope are not changed by recursive call
- flow of control passes back to **previous scope** once function call returns value

using the same variable names but they are different objects in separate scopes

ITERATION vs. RECURSION

```
def factorial_iter(n):  
    prod = 1  
    for i in range(1, n+1):  
        prod *= i  
    return prod
```

```
def factorial(n):  
    if n == 1:  
        return 1  
    else:  
        return n*factorial(n-1)
```

- recursion may be simpler, more intuitive
- recursion may be efficient from programmer POV
- recursion may not be efficient from computer POV

INDUCTIVE REASONING

- How do we know that our recursive code will work?
- `mult_iter` terminates because `b` is initially positive, and decreases by 1 each time around loop; thus must eventually become less than 1
- `mult` called with `b = 1` has no recursive call and stops
- `mult` called with `b > 1` makes a recursive call with a smaller version of `b`; must eventually reach call with `b = 1`

```
def mult_iter(a, b):  
    result = 0  
    while b > 0:  
        result += a  
        b -= 1  
    return result
```

```
def mult(a, b):  
    if b == 1:  
        return a  
    else:  
        return a + mult(a, b-1)
```

MATHEMATICAL INDUCTION

- To prove a statement indexed on integers is true for all values of n :
 - Prove it is true when n is smallest value (e.g. $n = 0$ or $n = 1$)
 - Then prove that if it is true for an arbitrary value of n , one can show that it must be true for $n+1$

EXAMPLE OF INDUCTION

■ $0 + 1 + 2 + 3 + \dots + n = (n(n+1))/2$

■ Proof: $\sum_{i=0}^n$

- If $n = 0$, then LHS is 0 and RHS is $0 \cdot 1/2 = 0$, so true
- Assume true for some k , then need to show that

$$0 + 1 + 2 + \dots + \underbrace{k}_{= n-1} + \underbrace{(k+1)}_{= n} = ((k+1)(k+2))/2 \quad \frac{(k+1)(k+1)}{2} = \frac{(k+1)(k+2)}{2}$$

- LHS is $k(k+1)/2 + (k+1)$ by assumption that property holds for problem of size k
- This becomes, by algebra, $((k+1)(k+2))/2$
- Hence expression holds for all $n \geq 0$

RELEVANCE TO CODE?

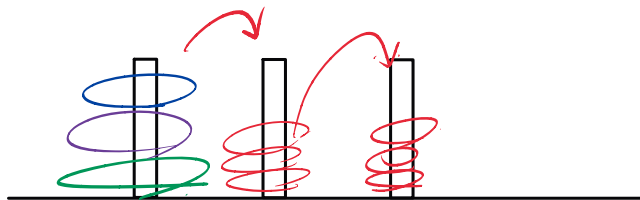
- Same logic applies

```
def mult(a, b):  
    if b == 1:  
        return a  
    else:  
        return a + mult(a, b-1)
```

- Base case, we can show that `mult` must return correct answer
- For recursive case, we can assume that `mult` correctly returns an answer for problems of size smaller than `b`, then by the addition step, it must also return a correct answer for problem of size `b`
- Thus by induction, code correctly returns answer

TOWERS OF HANOI

- The story:
 - 3 tall spikes
 - Stack of 64 different sized discs – start on one spike
 - Need to move stack to second spike (at which point universe ends)
 - Can only move one disc at a time, and a larger disc can never cover up a small disc



$$\begin{aligned}A_1 &= 1 \\ A_{n+1} &= 2A_n + 1 \\ \therefore A_n &= 2^n - 1\end{aligned}$$

TOWERS OF HANOI

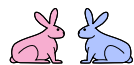
- Having seen a set of examples of different sized stacks, how would you write a program to print out the right set of moves?
- **Think recursively!**
 - Solve a smaller problem
 - Solve a basic problem
 - Solve a smaller problem

```
def printMove(fr, to):  
    print('move from ' + str(fr) + ' to ' + str(to))  
  
def Towers(n, fr, to, spare):  
    if n == 1:  
        printMove(fr, to)  
    else:  
        Towers(n-1, fr, spare, to)  
        Towers(1, fr, to, spare)  
        Towers(n-1, spare, to, fr)
```

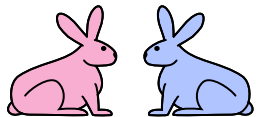
RECURSION WITH MULTIPLE BASE CASES

- Fibonacci numbers

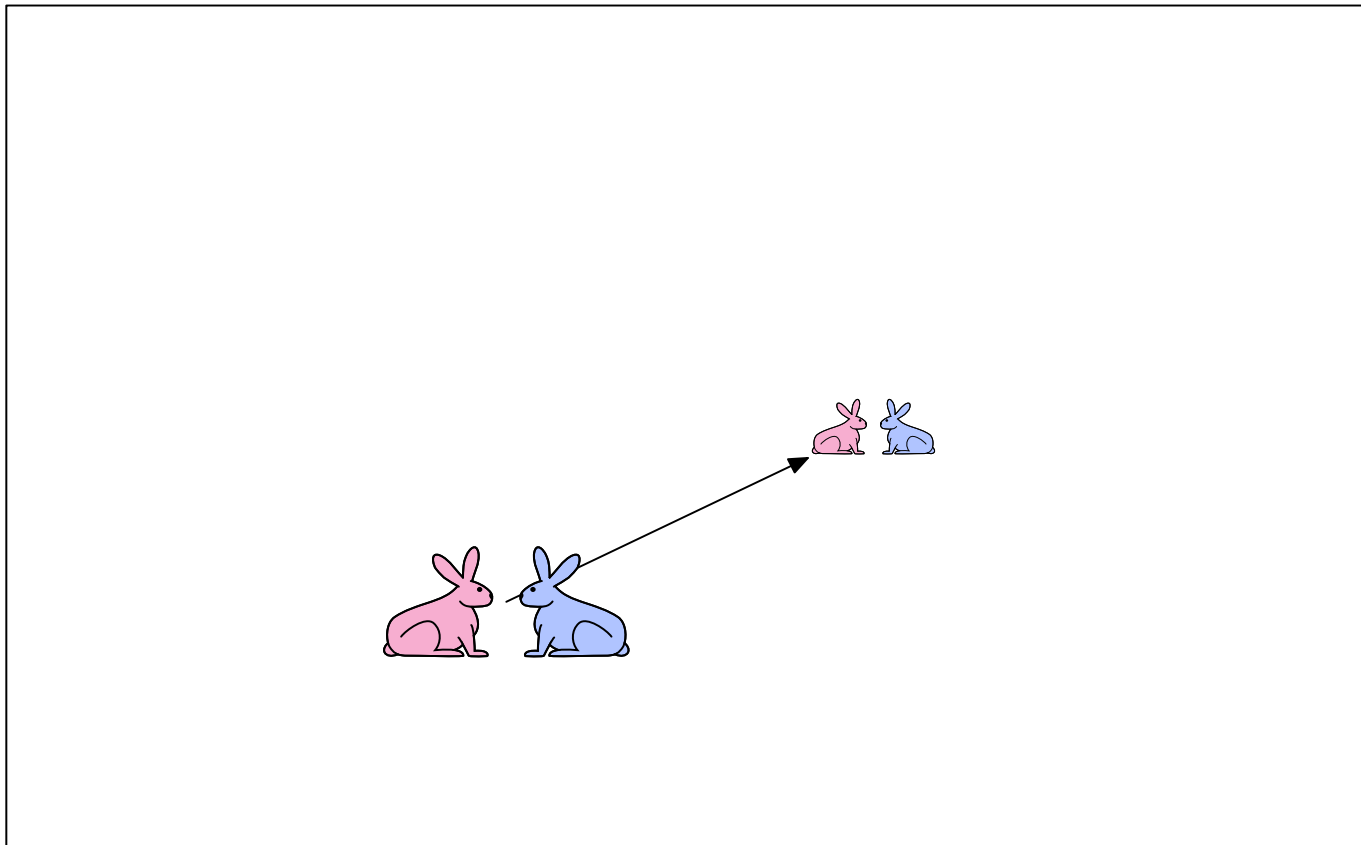
- Leonardo of Pisa (aka Fibonacci) modeled the following challenge
 - Newborn pair of rabbits (one female, one male) are put in a pen
 - Rabbits mate at age of one month
 - Rabbits have a one month gestation period
 - Assume rabbits never die, that female always produces one new pair (one male, one female) every month from its second month on.
 - How many female rabbits are there at the end of one year?



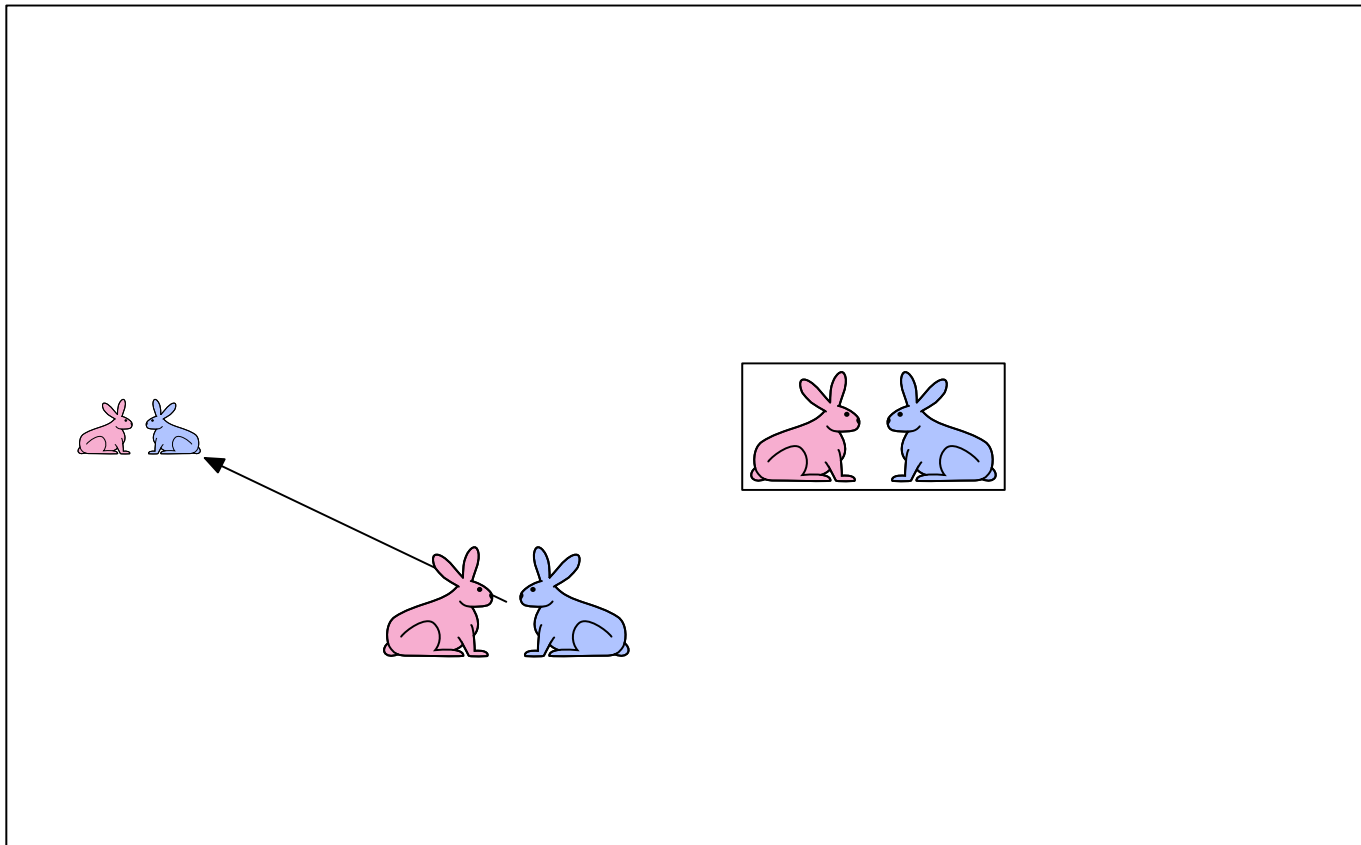
Demo courtesy of Prof. Denny Freeman and Adam Hartz



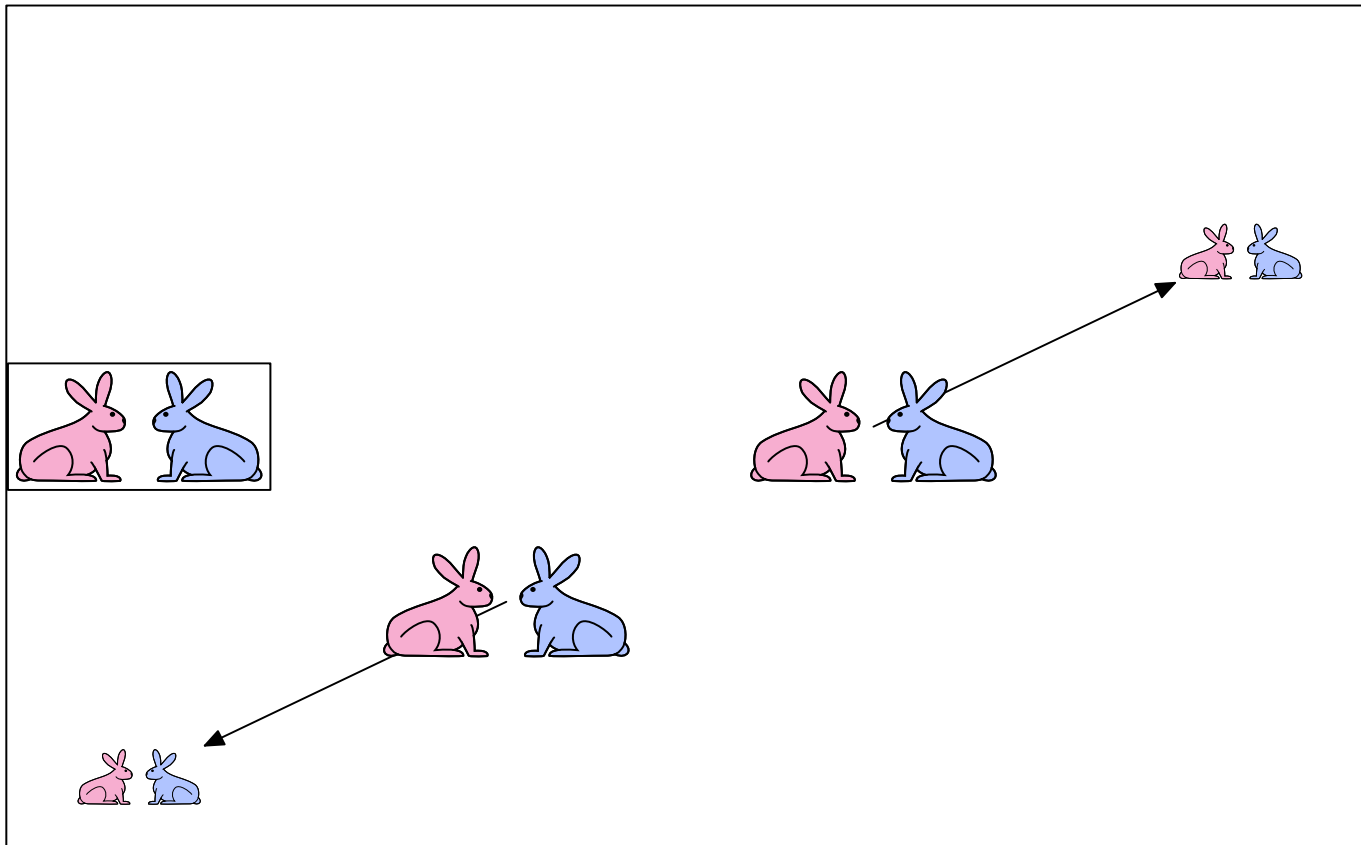
Demo courtesy of Prof. Denny Freeman and Adam Hartz



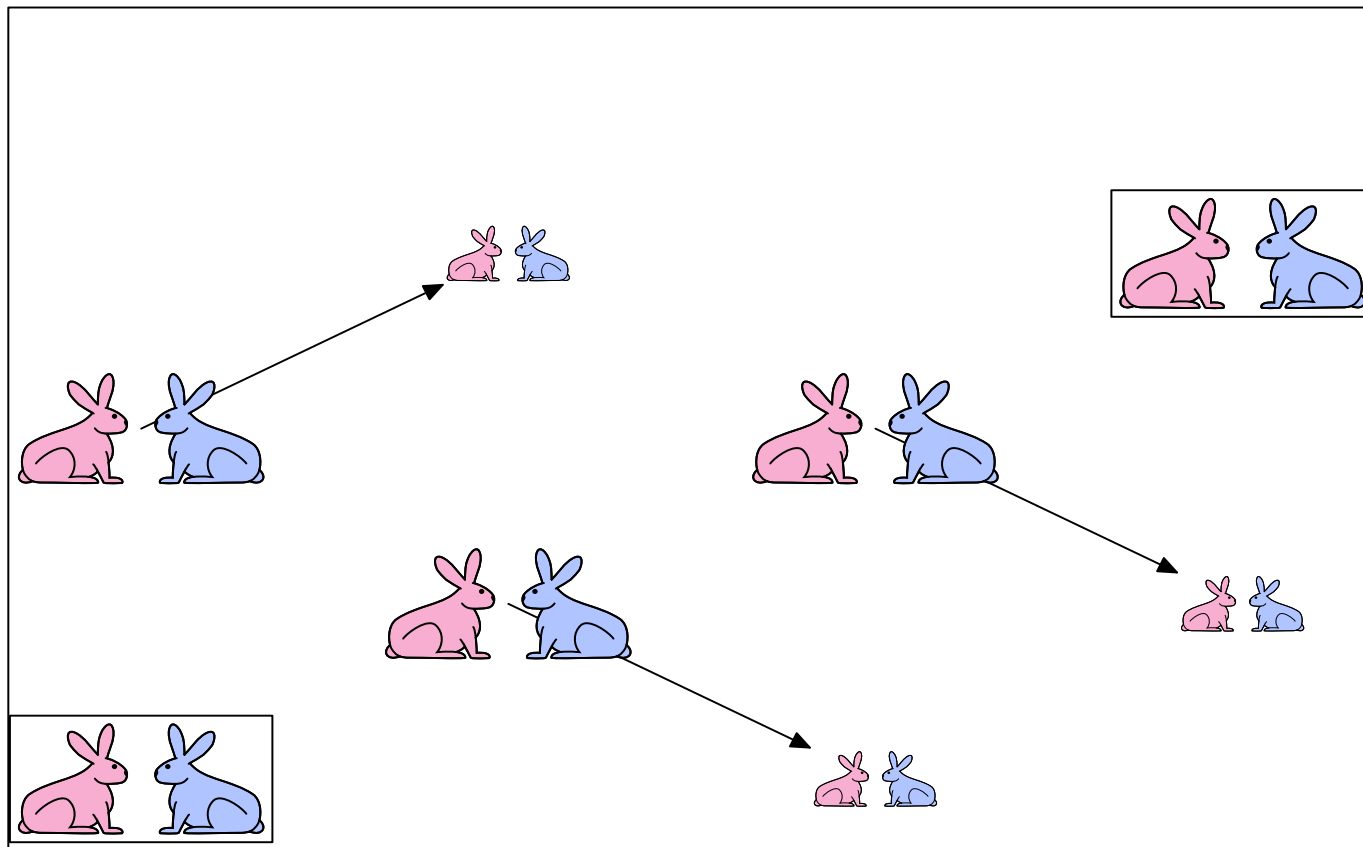
Demo courtesy of Prof. Denny Freeman and Adam Hartz



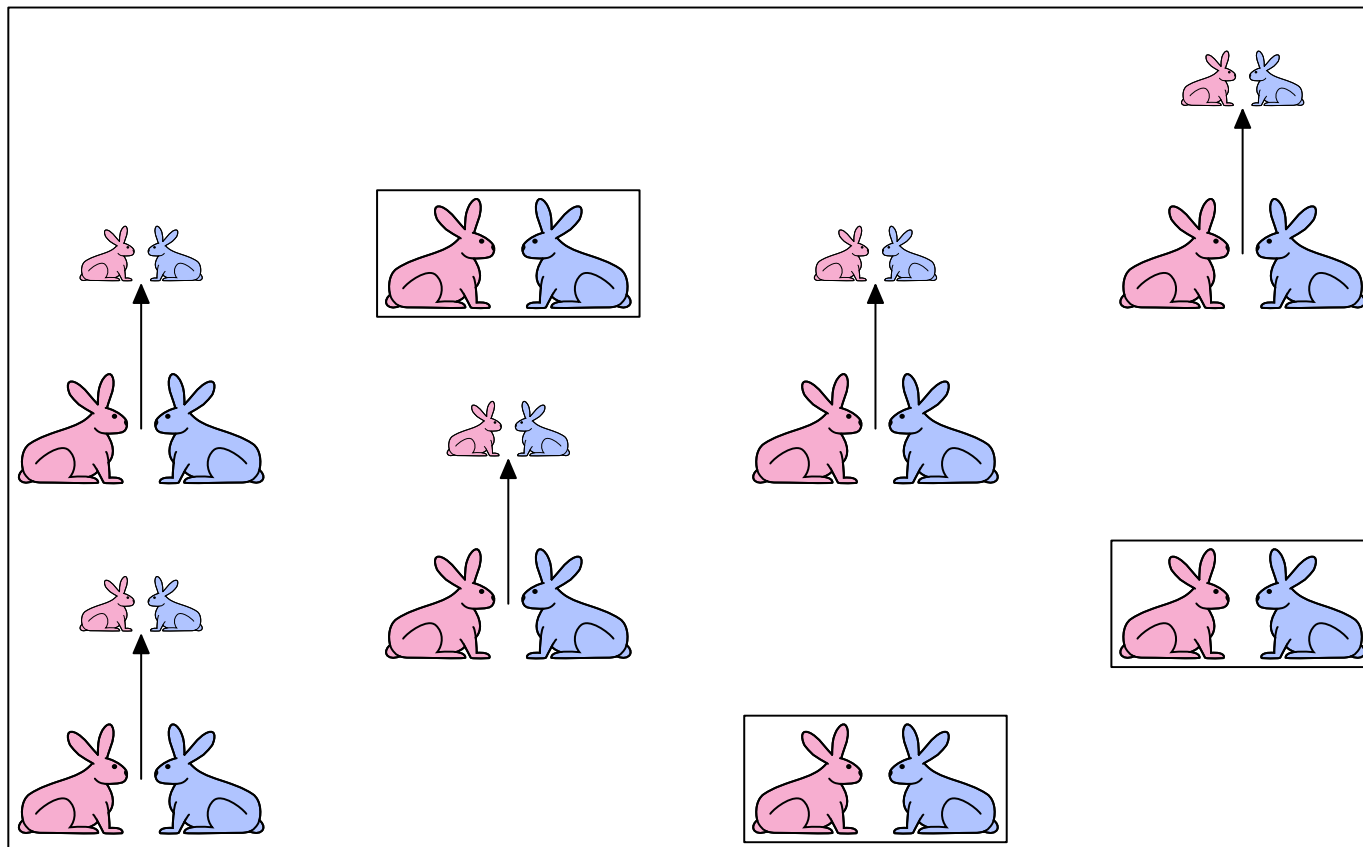
Demo courtesy of Prof. Denny Freeman and Adam Hartz



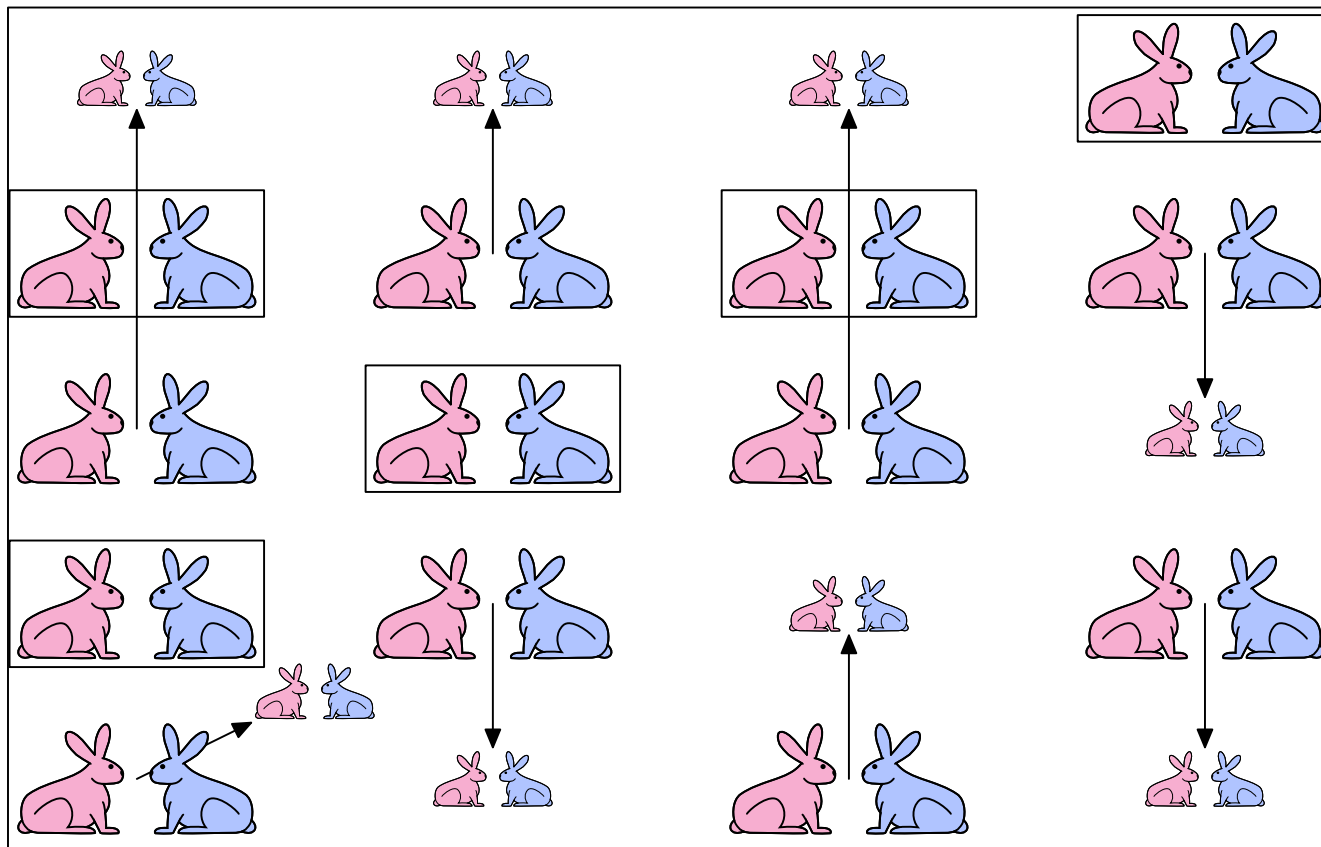
Demo courtesy of Prof. Denny Freeman and Adam Hartz

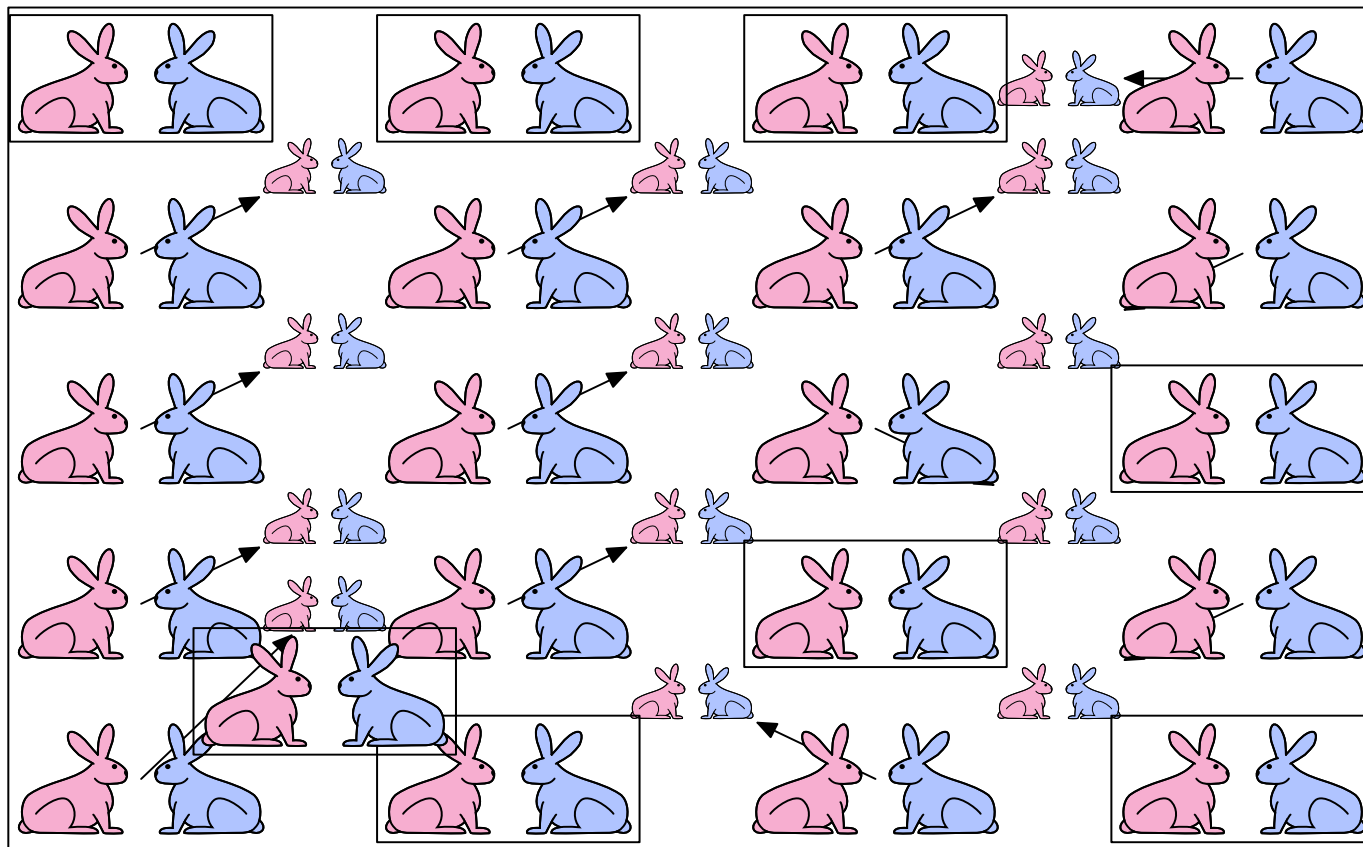


Demo courtesy of Prof. Denny Freeman and Adam Hartz

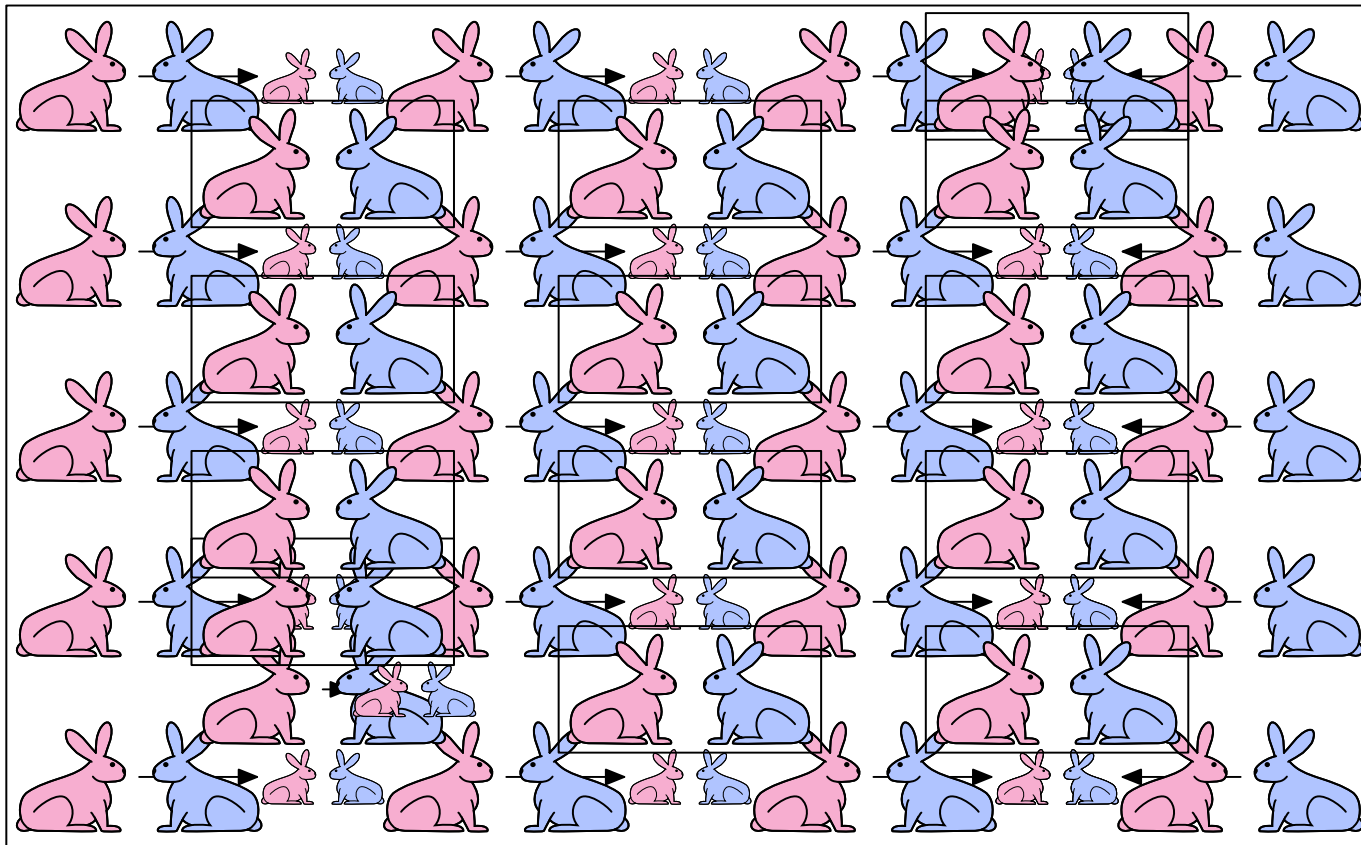


Demo courtesy of Prof. Denny Freeman and Adam Hartz





Demo courtesy of Prof. Denny Freeman and Adam Hartz



Demo courtesy of Prof. Denny Freeman and Adam Hartz

FIBONACCI

$$F_n = F_{n-1} + F_{n-2}$$
$$F_1 = F_2 = 1$$

After one month (call it 0) – 1 female

After second month – still 1 female (now pregnant)

After third month – two females, one pregnant, one not

In general, $\text{females}(n) = \text{females}(n-1) + \text{females}(n-2)$

- Every female alive at month $n-2$ will produce one female in month n ;
- These can be added those alive in month $n-1$ to get total alive in month n

Month	Females
0	1

FIBONACCI

- Base cases:

- Females(0) = 1 $\equiv F_1$
- Females(1) = 1 $\equiv F_2$

- Recursive case

- Females(n) = Females(n-1) + Females(n-2)

$$\boxed{n=2} \quad F(2) = F(1) + F(0) \quad \therefore F(2) = 2$$
$$= 2.$$

$$\boxed{n=3} \quad F(3) = F(2) + F(1) \quad \therefore F(3) = 3$$
$$= 3$$

⋮

FIBONACCI

```
def fib(x):  
    """assumes x an int >= 0  
        returns Fibonacci of x"""  
    if x == 0 or x == 1:  
        return 1  
    else:  
        return fib(x-1) + fib(x-2)  
                 $\equiv F(n-1)$        $\equiv F(n-2)$ 
```

RECURSION ON NON-NUMERICS

- how to check if a string of characters is a palindrome, i.e., reads the same forwards and backwards
 - “Able was I, ere I saw Elba” – attributed to Napoleon
 - “Are we not drawn onward, we few, drawn onward to new era?” – attributed to Anne Michaels



Image courtesy of [wikipedia](#), in the public domain.



By Larth_Rasnal (Own work) [GFDL (<https://www.gnu.org/licenses/fdl-1.3.en.htm>) or CC BY 3.0 (<https://creativecommons.org/licenses/by/3.0>)], via Wikimedia Commons.

SOLVING RECURSIVELY?

- First, convert the string to just characters, by stripping out punctuation, and converting upper case to lower case
- Then
 - Base case: a string of length 0 or 1 is a palindrome
 - Recursive case:
 - If first character matches last character, then is a palindrome if middle section is a palindrome

EXAMPLE

- 'Able was I, ere I saw Elba' → 'ablewasiereisawleba'
- `isPalindrome('ablewasiereisawleba')`
is same as
 - `'a' == 'a'` and
`isPalindrome('blewasiereisawleb')`

```

def isPalindrome(s):

    def toChars(s):
        s = s.lower()
        ans = ''
        for c in s:
            if c in 'abcdefghijklmnopqrstuvwxyz':
                ans = ans + c
        return ans

    def isPal(s):
        if len(s) <= 1:
            return True
        else:
            return s[0] == s[-1] and isPal(s[1:-1])

    return isPal(toChars(s))

```

DIVIDE AND CONQUER

- an example of a “divide and conquer” algorithm
- solve a hard problem by breaking it into a set of sub-problems such that:
 - sub-problems are easier to solve than the original
 - solutions of the sub-problems can be combined to solve the original

DICTIONARIES

HOW TO STORE STUDENT INFO

- so far, can store using separate lists for every info

```
names = ['Ana', 'John', 'Denise', 'Katy']
```

```
grade = ['B', 'A+', 'A', 'A']
```

```
course = [2.00, 6.0001, 20.002, 9.01]
```

- a **separate list** for each item
- each list must have the **same length**
- info stored across lists at **same index**, each index refers to info for a different person

HOW TO UPDATE/RETRIEVE STUDENT INFO

```
def get_grade(student, name_list, grade_list, course_list):  
    i = name_list.index(student)  
    grade = grade_list[i]  
    course = course_list[i]  
    return (course, grade)
```

- **messy** if have a lot of different info to keep track of
- must maintain **many lists** and pass them as arguments
- must **always index** using integers
- must remember to change multiple lists

A BETTER AND CLEANER WAY – A DICTIONARY

- nice to **index item of interest directly** (not always int)
- nice to use **one data structure**, no separate lists

A list

0	Elem 1
1	Elem 2
2	Elem 3
3	Elem 4
...	...

index

element

A dictionary

Key 1	Val 1
Key 2	Val 2
Key 3	Val 3
Key 4	Val 4
...	...

custom
index by
label

element

A PYTHON DICTIONARY

- store pairs of data
 - key
 - value

'Ana'	'B'
'Denise'	'A'
'John'	'A+'
'Katy'	'A'

custom
index by
label

element

my_dict = { } *empty dictionary*

grades = { 'Ana': 'B', 'John': 'A+', 'Denise': 'A', 'Katy': 'A' }

↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑
key1 val1 key2 val2 key3 val3 key4 val4

DICTIONARY LOOKUP

- similar to indexing into a list
- **looks up** the **key**
- **returns** the **value** associated with the key
- if key isn't found, get an error

'Ana'	'B'
'Denise'	'A'
'John'	'A+'
'Katy'	'A'

```
grades = {'Ana':'B', 'John':'A+', 'Denise':'A', 'Katy':'A'}
```

```
grades['John']      → evaluates to 'A+'
```

```
grades['Sylvan']    → gives a KeyError
```

DICTIONARY OPERATIONS

'Ana'	'B'
'Denise'	'A'
'John'	'A+'
'Katy'	'A'
'Sylvan'	'A'

```
grades = {'Ana': 'B', 'John': 'A+', 'Denise': 'A', 'Katy': 'A'}
```

- **add** an entry

```
grades['Sylvan'] = 'A'
```

- **test** if key in dictionary

```
'John' in grades      → returns True  
'Daniel' in grades   → returns False
```

- **delete** entry

```
del(grades['Ana'])
```

DICTIONARY OPERATIONS

'Ana'	'B'
'Denise'	'A'
'John'	'A+'
'Katy'	'A'

```
grades = {'Ana': 'B', 'John': 'A+', 'Denise': 'A', 'Katy': 'A'}
```

- get an **iterable that acts like a tuple of all keys** *no guaranteed order*
`grades.keys()` → returns `['Denise', 'Katy', 'John', 'Ana']`

- get an **iterable that acts like a tuple of all values**
`grades.values()` → returns `['A', 'A', 'A+', 'B']`

no guaranteed order

DICTIONARY KEYS and VALUES

■ values

- any type (**immutable and mutable**)
- can be **duplicates**
- dictionary values can be lists, even other dictionaries!

■ keys

- must be **unique**
- **immutable** type (`int`, `float`, `string`, `tuple`, `bool`)
 - actually need an object that is **hashable**, but think of as immutable as all immutable types are hashable
- careful with `float` type as a key

■ **no order** to keys or values!

```
d = {4:{1:0}, (1,3):"twelve", 'const':[3.14,2.7,8.44]}
```


list vs dict

- **ordered** sequence of **elements**
- look up elements by an integer index
- indices have an **order**
- index is an **integer**

- **matches** **“keys”** to **“values”**
- look up one item by another item
- **no order** is guaranteed
- key can be any **immutable** type

EXAMPLE: 3 FUNCTIONS TO ANALYZE SONG LYRICS

- 1) create a **frequency dictionary** mapping `str:int`
- 2) find **word that occurs the most** and how many times
 - use a list, in case there is more than one word
 - return a tuple `(list, int)` for `(words_list, highest_freq)`
- 3) find the **words that occur at least X times**
 - let user choose “at least X times”, so allow as parameter
 - return a list of tuples, each tuple is a `(list, int)` containing the list of words ordered by their frequency
 - IDEA: From song dictionary, find most frequent word. Delete most common word. Repeat. It works because you are mutating the song dictionary.

CREATING A DICTIONARY

```
def lyrics_to_frequencies(lyrics):  
    myDict = {}  
    for word in lyrics:  
        if word in myDict:  
            myDict[word] += 1  
        else:  
            myDict[word] = 1  
    return myDict
```

can iterate over list
can iterate over keys
in dictionary
update value
associated with key

USING THE DICTIONARY

```
def most_common_words(freqs):  
    values = freqs.values()  
    best = max(values)  
    words = []  
    for k in freqs:  
        if freqs[k] == best:  
            words.append(k)  
    return (words, best)
```

this is an iterable, so can
apply built-in function

can iterate over keys
in dictionary

LEVERAGING DICTIONARY PROPERTIES

```
def words_often(freqs, minTimes):  
    result = []  
    done = False  
    while not done:  
        temp = most_common_words(freqs)  
        if temp[1] >= minTimes:  
            result.append(temp)  
            for w in temp[0]:  
                del(freqs[w])  
        else:  
            done = True  
    return result
```

*can directly mutate
dictionary; makes it
easier to iterate*

```
print(words_often(beatles, 5))
```

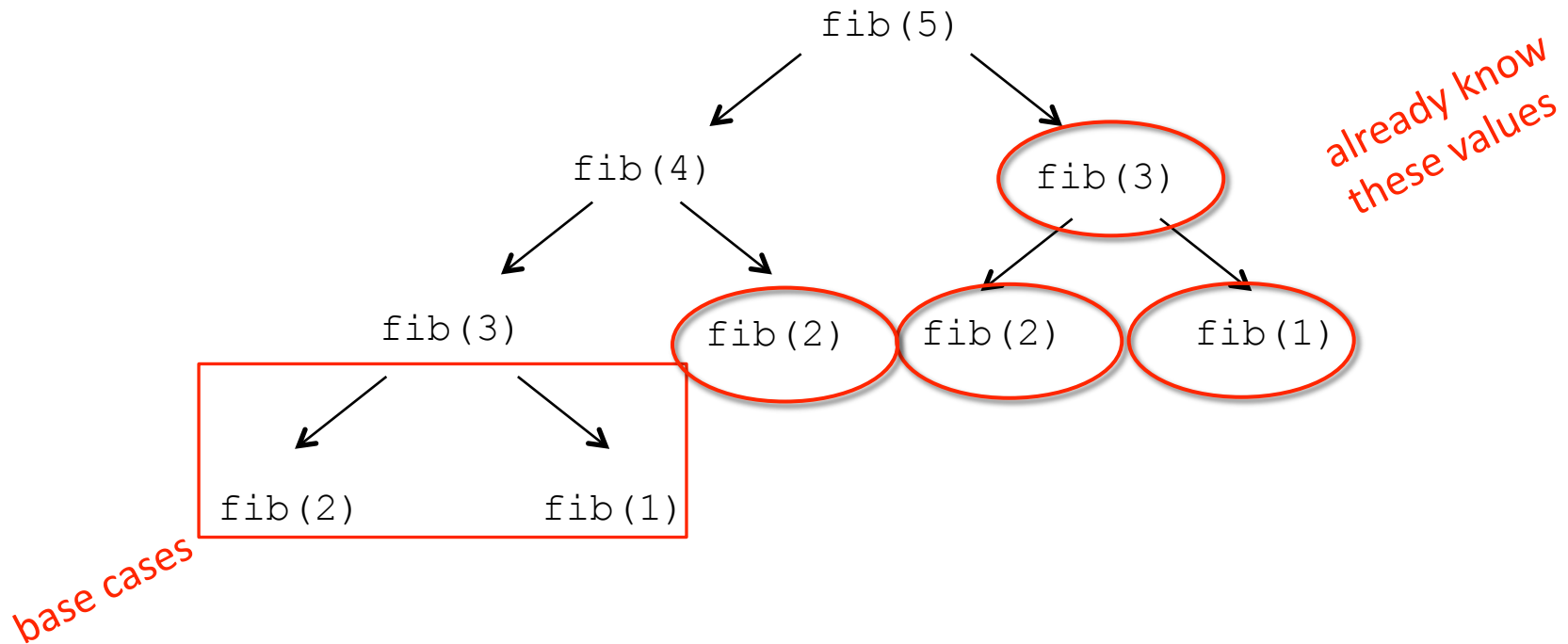
FIBONACCI RECURSIVE CODE

```
def fib(n):  
    if n == 1:  
        return 1  
    elif n == 2:  
        return 2  
    else:  
        return fib(n-1) + fib(n-2)
```

- two base cases
- calls itself twice
- this code is inefficient

INEFFICIENT FIBONACCI

$$\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$$



- **recalculating** the same values many times!
- could keep **track** of already calculated values

FIBONACCI WITH A DICTIONARY

```
def fib_efficient(n, d):  
    if n in d:  
        return d[n]  
    else:  
        ans = fib_efficient(n-1, d) + fib_efficient(n-2, d)  
        d[n] = ans  
        return ans  
  
d = {1:1, 2:2}  
print(fib_efficient(6, d))
```

Method sometimes
called "memoization"

Initialize dictionary
with base cases

- do a **lookup first** in case already calculated the value
- **modify dictionary** as progress through function calls

EFFICIENCY GAINS

- Calling `fib(34)` results in 11,405,773 recursive calls to the procedure
- Calling `fib_efficient(34)` results in 65 recursive calls to the procedure
- Using dictionaries to capture intermediate results can be very efficient
- But note that this only works for procedures without side effects (i.e., the procedure will always produce the same result for a specific argument independent of any other computations between calls)

MIT OpenCourseWare
<https://ocw.mit.edu>

6.0001 Introduction to Computer Science and Programming in Python
Fall 2016

For information about citing these materials or our Terms of Use, visit: <https://ocw.mit.edu/terms>.